# Installation & Execution Guide

**Project:** Cloud Log Monitoring System

**Course:** CS 623 - Cloud Computing

**Document:** Installation and Execution Instructions

**Team Members:**

Shreya Kumari (NET ID: fd6317)

Richa Sapre (NET ID: ee9498)

## Table of Contents

1. Prerequisites
2. Project Structure
3. Frontend Setup (Local Development)
4. Backend Setup (Local Development)
5. Google Cloud Platform Setup
6. Environment Variables
7. Running the Application
8. Testing the Application
9. Troubleshooting

## 1. Prerequisites

Before you begin, ensure you have the following installed on your machine:

| Software | Version | Download Link |
|---|---|---|
| Node.js | 18.x or higher | https://nodejs.org/ |
| npm | 9.x or higher | (Comes with Node.js) |
| Python | 3.10 or higher | https://python.org/ |
| pip | Latest | (Comes with Python) |
| Google Cloud SDK | Latest | https://cloud.google.com/sdk |
| Git | Latest | https://git-scm.com/ |

## Verify Installations

```
# Check Node.js version node --version # Check npm version npm --
version # Check Python version python --version # Check pip
version pip --version # Check Google Cloud SDK gcloud --version
```

## 2. Project Structure

```
cloud-monitoring-system/
├── backend/
│   ├── api.py                 # Flask REST API
│   ├── function.py            # Cloud Function code
│   ├── requirements.txt       # Python dependencies
│   └── README.txt             # Backend documentation
├── src/
│   ├── App.tsx               # Main React component
│   ├── main.tsx              # Entry point
│   ├── types.ts              # TypeScript types
│   ├── styles.css            # Global styles
│   ├── components/
│   │   ├── Header.tsx        # Header component
│   │   ├── StatsCards.tsx    # Statistics display
│   │   ├── TrendsChart.tsx   # Chart visualization
│   │   ├── Filters.tsx       # Filter controls
│   │   ├── LogTable.tsx      # Log data table
│   │   ├── SeverityBadge.tsx # Severity indicator
│   │   ├── AddLogModal.tsx   # Add log form
│   │   ├── AlertsPanel.tsx   # Alert management
│   │   └── AlertToaster.tsx  # Notifications
│   └── lib/
│       ├── api.ts            # API client
│       └── alerts.ts         # Alert logic
├── docs/                      # Documentation
├── package.json              # Node.js dependencies
├── vite.config.ts            # Vite configuration
├── tailwind.config.js        # Tailwind CSS config
├── tsconfig.json             # TypeScript config
├── firebase.json             # Firebase hosting config
└── README.md                 # Project readme
```

## 3. Frontend Setup (Local Development)

### 1 Navigate to Project Directory

```
cd cloud-monitoring-system
```

## 2 Install Dependencies

```
npm install
```

This will install the following packages:

- **react**, **react-dom** - UI library
- **recharts** - Charting library
- **lucide-react** - Icons
- **date-fns** - Date formatting
- **clsx** - CSS class management
- **tailwindcss** - CSS framework
- **typescript** - Type checking
- **vite** - Build tool

## 3 Start Development Server

```
npm run dev
```

✓ **Success!** The frontend will be available at: **http://localhost:5173**

## 4. Backend Setup (Local Development)

### 1 Navigate to Backend Directory

```
cd backend
```

### 2 Create Virtual Environment (Recommended)

```
# Create virtual environment python -m venv venv # Activate
virtual environment # On macOS/Linux: source venv/bin/activate #
On Windows: venv\Scripts\activate
```

### 3 Install Python Dependencies

```
pip install -r requirements.txt
```

This will install:

- **flask** - Web framework
- **flask-cors** - CORS support
- **google-cloud-firestore** - Firestore client
- **google-cloud-pubsub** - Pub/Sub client
- **google-cloud-logging** - Logging client
- **requests** - HTTP library

### 4 Set Environment Variables

```
# On macOS/Linux: export GCP_PROJECT=your-gcp-project-id export
GOOGLE_APPLICATION_CREDENTIALS=/path/to/service-account-key.json
# On Windows (Command Prompt): set GCP_PROJECT=your-gcp-project-
id set GOOGLE_APPLICATION_CREDENTIALS=C:\path\to\service-account-
key.json
```

### 5 Run the API Server

```
python api.py
```

✓ **Success!** The API will be available at: **http://localhost:8080**

## 5. Google Cloud Platform Setup

⚠ **Important:** These steps are required if you want to deploy to GCP or use GCP services locally.

### 1 Authenticate with Google Cloud

```
gcloud auth login gcloud config set project YOUR_PROJECT_ID
```

### 2 Enable Required APIs

```
gcloud services enable run.googleapis.com gcloud services enable pubsub.googleapis.com gcloud services enable cloudfunctions.googleapis.com gcloud services enable firestore.googleapis.com
```

### 3 Create Pub/Sub Topic

```
gcloud pubsub topics create log-topic
```

### 4 Deploy Cloud Function

```
gcloud functions deploy process-log-function \ --runtime python310 \ --trigger-topic log-topic \ --entry-point process_log \ --source ./backend \ --set-env-vars SENDER_EMAIL=your-email@gmail.com,RECEIVER_EMAIL=alert@gmail.com,EMAIL_PASSWORD=your-app-password
```

### 5 Deploy API to Cloud Run

```
gcloud run deploy log-monitoring-api \ --source ./backend \ --region us-central1 \ --allow-unauthenticated \ --set-env-vars GCP_PROJECT=YOUR_PROJECT_ID
```

## 6 Build and Deploy Frontend

```
# Build the frontend npm run build # Deploy to Firebase Hosting
firebase deploy --only hosting
```

## 6. Environment Variables

### Frontend Environment Variables (.env file)

```
# Create .env file in project root # API Base URL (Cloud Run endpoint) VITE_API_BASE_URL=https://your-api-url.run.app # Auto-refresh interval in milliseconds VITE_POLL_INTERVAL_MS=10000 # Alert webhook URL (optional - for Slack integration) VITE_ALERT_WEBHOOK_URL=https://hooks.slack.com/services/xxx # Alert cooldown in minutes VITE_ALERT_COOLDOWN_MIN=5
```

### Backend Environment Variables

| Variable | Description | Required |
|---|---|---|
| GCP_PROJECT | Google Cloud Project ID | Yes |
| GOOGLE_APPLICATION_CREDENTIALS | Path to service account key JSON | Yes (local) |
| SENDER_EMAIL | Gmail address for sending alerts | For alerts |
| RECEIVER_EMAIL | Email to receive alerts | For alerts |
| EMAIL_PASSWORD | Gmail App Password | For alerts |
| PORT | API server port (default: 8080) | No |

## 7. Running the Application

### Option A: Local Development (Both Frontend & Backend)

**Terminal 1 - Backend:**

```
cd backend source venv/bin/activate export GCP_PROJECT=your-project-id python api.py
```

**Terminal 2 - Frontend:**

```
cd cloud-monitoring-system npm run dev
```

### Option B: Using Deployed Cloud Services

```
# Just run the frontend - it connects to Cloud Run API npm run dev
```

✓ **Application Running!**

Frontend: http://localhost:5173

Backend API: http://localhost:8080 (or Cloud Run URL)

# 8. Testing the Application

## Test API Endpoints

**Add a Log Entry:**

```
curl -X POST http://localhost:8080/api/logs \ -H "Content-Type: application/json" \ -d '{"severity": "ERROR", "message": "Test error log", "source": "test-service"}'
```

**Get All Logs:**

```
curl http://localhost:8080/api/logs
```

**Get Logs with Filter:**

```
curl "http://localhost:8080/api/logs?severity=ERROR&limit=10"
```

**Get Statistics:**

```
curl http://localhost:8080/api/stats
```

## Test Dashboard Features

1. Open http://localhost:5173 in your browser
2. Click "Add Log" to create a new log entry
3. Use filters to search logs by severity, source, or message
4. Check the chart for log trends over time
5. Click "Alerts" to create alert rules
6. Verify browser notifications work (allow when prompted)

# 9. Troubleshooting

| Issue | Solution |
|---|---|
| npm install fails | Delete node_modules folder and package-lock.json, then run npm install again |
| CORS errors in browser | Ensure Flask-CORS is installed and configured. Check API URL is correct. |
| Firestore connection fails | Verify GOOGLE_APPLICATION_CREDENTIALS points to valid service account key |
| Pub/Sub permission denied | Ensure service account has Pub/Sub Publisher role |
| Email alerts not sending | Verify Gmail App Password is correct. Enable "Less secure apps" or use App Password. |
| Port already in use | Kill existing process: `lsof -i :8080` then `kill -9 PID` |
| TypeScript errors | Run `npm run build` to check for type errors |

**Need Help?**
Check the backend/README.txt for additional backend documentation and API reference.

---

**End of Installation Guide**